

Real-Time FPGA Implementation of a QPSK Modulator and Demodulator with Simulated Channel

Anonymous CVPR submission

Paper ID *****

Abstract

Quadrature Phase Shift Keying (QPSK) is a digital modulation technique which transmits two bits per symbol without additional bandwidth. This project implements a complete, real-time QPSK transmitter and receiver entirely on a Spartan-7 FPGA (Boolean Board, XC7S50). The system performs bit-to-symbol mapping, carrier generation, quadrature mixing, and matched low-pass filtering, and includes a simulated AWGN channel with live-adjustable noise level for testing. System performance is validated through bit-error-rate (BER) measurement and live constellation visualization streamed over UART. This report defines the problem statement, reviews relevant literature on QPSK theory and existing FPGA-based QPSK implementations, and outlines the architecture and verification plan for the mid-term and final phases. Carrier recovery via a Costas loop, symbol timing recovery via the Gardner algorithm, and a coded-versus-uncoded BER comparison are being considered as optional extensions if time permits.

1. Introduction

Modern communication systems rely heavily on digital modulation to move information reliably and efficiently over a band-limited channel. Digital modulation schemes vary properties of a carrier, such as its amplitude, phase, or frequency, to represent information while making efficient use of the available communication bandwidth.

Quadrature Phase Shift Keying (QPSK) is one such scheme. Unlike Binary PSK, which uses two carrier phases to represent one bit per symbol, QPSK uses four equally spaced carrier phases, so that each transmitted symbol represents two bits at once [2]. This is conveniently described using an in-phase (I) and quadrature (Q) component, which together represent the two bits of each transmitted symbol. The transmitted signal can therefore be represented as a combination of cosine and sine carriers at the same frequency, with the I and Q components determining the cor-

responding symbol.

Field-Programmable Gate Arrays (FPGAs) provide re-configurable digital hardware rather than a fixed instruction-execution pipeline: operations such as carrier generation, multiplication, and filtering can each be realized as dedicated, concurrently operating hardware blocks. This matters for a real-time communication system, where all stages of modulation and demodulation must run continuously at the signal's own rate. Prior work has already demonstrated that a complete QPSK modulator and demodulator can be realized on FPGA hardware [1, 4], which motivates implementing our system on FPGA rather than restricting it to a Python/MATLAB simulation alone.

Narrowing from this general motivation to our specific system, this project implements a complete QPSK transmit-receive chain in which a bit stream is first mapped to QPSK symbols, carriers are generated by an NCO, the symbols are modulated onto these carriers, and the resulting signal is passed through a simulated AWGN channel. At the receiver, the signal is coherently demodulated, passed through an FIR filter, and a symbol decision block recovers the original bit stream. Each of these stages is implemented as a distinct hardware block on the Spartan-7 FPGA (Boolean Board, XC7S50).

To validate this system, we follow a staged verification approach: a software reference model is developed first, followed by a fixed-point version of the same algorithm, and finally the FPGA/RTL implementation itself. At each stage, the implementation's behavior is compared against the reference model, and the overall system is evaluated using bit-error-rate (BER), constellation diagrams, behavior under varying noise conditions, and FPGA implementation characteristics such as resource utilization.

2. Literature Review

The theoretical foundation for this project is Proakis and Salehi [5], whose treatment of digital modulation and optimum receiver design for additive white Gaussian noise (AWGN) channels provides the signal-space and constella-

tion basis for QPSK, along with the standard framework for coherent detection and error-rate analysis. This underlies the QPSK constellation, matched filtering, and BER evaluation used throughout our design, without being an FPGA implementation reference itself.

The most directly relevant prior FPGA implementation is Popescu *et al.* [4], who implement a complete QPSK modulator, channel, and demodulator rather than a modulator alone, using two Spartan-3E boards with ROM-generated sine/cosine carriers and coherent multiply-and-threshold detection at the receiver. This establishes that a complete FPGA-based QPSK transmit-receive chain is practical on modest hardware, and is the closest architectural match to our own single-board system.

On the modulator side, Al Safi and Bazuin [1] focus on carrier generation, producing the sine/cosine waveforms needed for BPSK/QPSK using a single LUT addressed by a reverse-accumulator technique to reduce memory — directly relevant to our own phase-accumulator-driven NCO. Pareek [3] shows that this I/Q-and-multiplier structure is not the only option, comparing it against a digitized-carrier approach that uses a 4:1 multiplexer to select one of four pre-computed phase-shifted carriers directly from the input bit pair. We adopt the conventional I/Q route used by the first two papers, for its direct correspondence between modulation and demodulation stages.

On the demodulator side, Zhang *et al.* [7] present an all-digital receiver built from digital down-conversion (NCO, mixer, low-pass filter), a four-branch Costas loop for carrier synchronization, and a digital-phase-locked-loop bit synchronizer. This is useful primarily as a contrast: it shows what a synchronization-capable receiver requires, which lets us state plainly that our baseline demodulator instead assumes a fixed, known carrier phase and symbol timing, deferring carrier/timing recovery to the optional-extension scope described in section 1.

Deepika *et al.* [6] demonstrate that coherent I/Q multiplication is not the only demodulation strategy either, proposing a comparator-based QPSK demodulator built with model-based design and implemented on a Zynq-7000 (ZED) board, with reported hardware resource utilization. This confirms that alternative, lower-complexity demodulator architectures exist, though our project uses explicit I/Q demodulation with locally generated references, FIR filtering, and symbol decision rather than a comparator-based scheme.

Taken together, the reviewed literature establishes the theoretical basis for QPSK and AWGN detection, demonstrates a complete two-board FPGA QPSK system, and covers multiple approaches to both carrier generation/modulation and demodulation, ranging from ROM/accumulator carrier generation to comparator-based and synchronization-capable receivers. Our contribution

is not a new modulation or demodulation algorithm, but the integration of established, conventional techniques into one compact, single-FPGA system with a digitally simulated, live-adjustable AWGN channel, BER measurement, and UART-streamed constellation visualization.

3. Next Steps

Having established the problem statement and reviewed relevant prior work, the next phase of this project is to design, implement, and verify the core QPSK system described in section 1. We plan to follow a staged verification approach: a software reference model will be developed first in Python, followed by a fixed-point version of the same algorithm, and finally the FPGA/RTL implementation on the Boolean Board. At each stage, the implementation's output will be compared against the previous stage to isolate errors early, rather than debugging the algorithm and the hardware simultaneously.

Concretely, for the mid-term report we aim to have a working modulator and a demodulator that assumes a fixed, known carrier phase, verified in simulation, along with the simulated AWGN channel with its live-adjustable noise level. For the final report, we aim to complete the bit-error-rate measurement pipeline and the live constellation visualization streamed over UART, and to demonstrate the full system on the Boolean Board.

If time permits after this core system is verified and working, we will consider the optional extensions introduced in section 1: carrier phase recovery via a Costas loop, symbol timing recovery via the Gardner algorithm, and a coded-versus-uncoded BER comparison. These remain secondary to completing and validating the core system.

References

- [1] Ameen Al Safi and Bradley Bazuin. Fpga based implementation of bpsk and qpsk modulators using address reverse accumulators. In *2016 IEEE 7th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, pages 1–6, 2016. 1, 2
- [2] ECE, IIT Kharagpur. Quaternary phase shift keying (qpsk) modulation. NPTEL, Module 5, Lesson 25. Version 2. 1
- [3] Vishakha Pareek. A novel implementation of qpsk modulator on fpga. In *2016 IEEE Students' Conference on Electrical, Electronics and Computer Science (SCEECS)*, pages 1–4, 2016. 2
- [4] S.O. Popescu, A.S. Gontean, and D. Ianchis. Implementation of a qpsk system on fpga. In *2011 IEEE 9th International Symposium on Intelligent Systems and Informatics*, pages 365–370, 2011. 1, 2
- [5] John G. Proakis and Masoud Salehi. *Communication Systems Engineering*. Prentice Hall / Pearson Education, 2nd edition, 2002. 1
- [6] Deepika R., Sourabh K.S., and Manikandan J. Design and implementation of a novel qpsk demodulator for fpga based sys-

216		270
217	tem design. In <i>2019 IEEE International WIE Conference on</i>	271
218	<i>Electrical and Computer Engineering (WIECON-ECE)</i> , pages	272
219	1–4, 2019. 2	273
220	[7] Huiying Zhang, Ting Zhao, and Ruijing Xie. Research of all	274
221	digital demodulation technology for qpsk. 01 2015. 2	275
222		276
223		277
224		278
225		279
226		280
227		281
228		282
229		283
230		284
231		285
232		286
233		287
234		288
235		289
236		290
237		291
238		292
239		293
240		294
241		295
242		296
243		297
244		298
245		299
246		300
247		301
248		302
249		303
250		304
251		305
252		306
253		307
254		308
255		309
256		310
257		311
258		312
259		313
260		314
261		315
262		316
263		317
264		318
265		319
266		320
267		321
268		322
269		323